

Name: _____

Write your name and computing ID above. Write your computing ID at the top of each page in case pages get separated. Sign the honor pledge below.

Generally, we will not answer questions about the exam during the exam time. If you think a question is unclear and requires additional information to answer, please explain how in your answer. For multiple choice questions, write a * next to the relevant option(s) along with your explanation.

On my honor as a student I have neither given nor received aid on this exam.

1. Consider the following C code. Assume all necessary includes are present but not shown, that all state is correctly initialized, and that `PlaceMeetingIn()` and/or `CloseRoom()` are called from separate threads in the same program.

```

1  struct Room {
2      pthread_mutex_t lock;
3      int open_times[NUM_TIMES];
4      struct Meeting *meetings[NUM_TIMES];
5  };
6  struct Meeting {
7      pthread_mutex_t lock;
8      struct Room *room;
9      int time;
10 };
11
12 int PlaceMeetingIn(struct Meeting *meeting, struct Room *new_room) {
13     int successful = 0;
14     pthread_mutex_lock(&meeting->lock);
15     struct Room *old_room = meeting->room;
16     if (old_room != NULL) {
17         pthread_mutex_lock(&old_room->lock);
18         meeting->room = NULL;
19         old_room->meetings[meeting->time] = NULL;
20         pthread_mutex_unlock(&old_room->lock);
21     }
22     pthread_mutex_lock(&new_room->lock);
23     if (new_room->meetings[meeting->time] == NULL) {
24         new_room->meetings[meeting->time] = meeting;
25         meeting->room = new_room;
26         successful = 1;
27     }
28     pthread_mutex_unlock(&new_room->lock);
29     pthread_mutex_unlock(&meeting->lock);
30     return successful;
31 }
32
33 void CloseRoom(struct Room *room, int start, int end) {
34     pthread_mutex_lock(&room->lock);
35     for (int time = start; time < end; time += 1) {
36         struct Meeting *meeting = room->meetings[time];
37         if (meeting) {
38             pthread_mutex_lock(&meeting->lock);
39             meeting->room = NULL;
40             pthread_mutex_unlock(&meeting->lock);
41         }
42         room->open_times[time] = 0;
43     }
44     pthread_mutex_unlock(&room->lock);
45 }

```

- (a) (6 points) A deadlock is possible in this code. Give an example of how a deadlock could occur because of simultaneous calls to `PlaceMeetingIn` and/or `CloseRoom` in the above code.

Solution: if meeting M is in room R1, `PlaceMeetingIn(M, R2)`, `CloseRoom(M, ...)` with a range overlapping the meeting time

- (b) (6 points) In a misguided attempt to avoid a deadlock, a programmer removes the lock and corresponding unlock of `&old_room->lock` *[added after original exam: and `&new_room->lock`]* from `PlaceMeetingIn`. Briefly give an example of a likely way in which this could produce an incorrect result from two simultaneous `PlaceMeetingIn` calls.

Solution: my original intended answer was that there was a race between the two settings of `meetings[time]`, but that doesn't really work with the `== NULL` check. So we dropped this question. With the updated version of the question, we can have two `PlaceMeetingIns` think they succeed at filling the same location.

- (c) Briefly describe two ways to edit the above code to prevent the deadlock you described in part (a) without allowing for it to produce incorrect results.

- i. (3 points) Method 1:

Solution:

- ii. (3 points) Method 2:

Solution:

2. Suppose a system has:

- 4096-byte (2^{12} -byte) pages
- 48-bit virtual addresses
- 36-bit physical addresses
- 8-byte page table entries
- a 4-level page table structure with 512 entries in page tables at each level
- a 16-entry, 2-way TLB
- a 1024-set, 64-byte-block, direct-mapped data cache

Without a TLB, a program reading 4 bytes of data at virtual address 0x123456 triggers the following memory accesses:

purpose	physical address
read 1st level page table entry	0x20000
read 2nd level page table entry	0x30000
read 3rd level page table entry	0x40000
read 4th level page table entry	0x50090
read requested data	0x60456

Consider what will happen if there was a TLB and this read caused a **TLB miss**.

- (a) (3 points) From which physical address(es) did any information stored in the TLB come from as a result of the read?

Solution: from physical address 0x50090 only

- (b) (3 points) What set(s) of the TLB is any information as a result of the access stored in?

Solution: set index 3

3. Consider a system with 64-bit virtual addresses, 16384-byte (2^{14} byte) pages, and a 3-level page table with 2048 (2^{11}) 8-byte entries in tables at each level.
- (a) (6 points) Complete the function below to extract the index into the *second-level* page table.

```
unsigned long SecondLevelIndex(unsigned long virtual_address) {  
  
    unsigned long value_one = virtual_address >> (11+14);  
  
    unsigned long value_two = value_one & (0x7FF);  
  
    return value_two;  
}
```

>> is the right-shift operator, and & is the bitwise and operator.

- (b) (6 points) Assume the page table starts completely empty. These two events happen: *four extra 0s at the end of removed from below since original exam; we dropped this question on the original exam because exceeding the virtual address width was trickier than we liked*
- A. Allocate 0x30D07A9A0000
 - B. Allocate 0x30D4E29A0000

What is a virtual address that if allocated next will cause exactly two page tables to be allocated?

Solution:

4. Consider an L1 data cache with the following attributes:

- 64-byte (2^6 byte) cache blocks
- 4 sets
- 2 ways
- a write-back policy
- a write-allocate policy
- an LRU replacement policy

Suppose the cache is on a system with 32-bit physical addresses.

- (a) (10 points) Complete the table for the data and metadata the entire cache stores. (You may not need all lines. You may leave blanks as unsimplified arithmetic expressions.)

purpose	size (bits)
data	$64\text{B} \times 8\text{bit/B} \times 8\text{blocks} = 4096$
valid bits	8
dirty bits	8
tag bits	$8 \times (32 - 6 - 2) = 192$
LRU bits	4

- (b) (10 points) Suppose this cache is originally empty and then it services 5 accesses. While servicing those accesses, the cache writes 64 bytes to memory and reads 192 bytes (64×3 bytes) from memory and ends up with exactly two blocks of data stored, all in the set with index 0.

Fill in a possible sequence of 1-byte accesses that could have occurred: *one possible answer shown*

type (read or write)	address		
	tag	index	offset
read	1	0	any
write	1	0	any
read	2	0	any
read	3	0	any
read	2	0	any

5. A single process is running on a system. The following sequence of memory accesses occurs in this order:

- A. Read one byte at 0x09E (binary: 0 1001 1110)
- B. Read one byte at 0x016 (binary: 0 0001 0110)
- C. Read one byte at 0x1BD (binary: 1 1011 1101)
- D. Read one byte at 0x09B (binary: 0 1001 1011)

- (a) (6 points) Design a direct-mapped cache that
- has a block size of **at most** 128 bytes, and
 - is **no larger than** 500 bytes, and
 - ensures that read D will be a cache hit

Block size: 8 bytes

Number of sets: 4

if block size 8, at least 8 sets, at most 32 sets; if block size 16, exactly 16 sets; if block size 32, exactly 8 sets; block size 64 is not possible.

- (b) (6 points) Design a set associative cache that
- has a block size of **at least** 8 bytes, and
 - has **at least** two ways, and
 - ensures that read D will be a cache miss

one possible answer

Block size: 16 bytes

Number of sets: 2

Number of ways: 2

6. Consider the following C code:

```
1 void *two(void*);
2
3 pthread_t two_thr;
4 int global = 5;
5
6 void *one(void *argument) {
7     int *p;
8     p = (int*) argument;
9     *p = global + 10;
10    pthread_create(&two_thr, NULL, two, p);
11    return &global;
12 }
13
14 void *two(void *argument) {
15     int *q;
16     q = (int*) argument;
17     printf("A\n");
18     *q += 10;
19     return NULL;
20 }
21
22 int main() {
23     pthread_t one_thr;
24     int x = 20;
25     pthread_create(&one_thr, NULL, one, &x);
26     int *r;
27     pthread_join(one_thr, &r);
28     printf("B\n");
29     pthread_join(two_thr, NULL);
30     printf("%d\n", x);
31     printf("%d\n", *r);
32     return 0;
33 }
```

For the following questions, assume no pthread functions fail and that all relevant header files are included.

- (a) (3 points) The above code tries to print out both A and B. Which of the following are possible ways that output could appear? **Select all that apply.**
- ☒ A followed by B
 - ☒ B followed by A
 - ☐ just B (the statement printing A might not be reached)
 - ☐ just A (the statement printing B might not be reached)
- (b) (3 points) The line `*q += 10`, if it runs successfully, modifies _____.
- ☒ a value on the stack of the thread that runs `main()`
 - ☐ a value on the stack of the thread that runs `one()`
 - ☐ a value on the stack of the thread that runs `two()`
 - ☐ a value on the heap
 - ☐ a value in the region used to store global variables
- (c) (3 points) The line `printf("%d\n", *r);` outputs _____. *was written without \n on original exam*
- ☐ a value on the stack of the thread that runs `main()`
 - ☒ a value in the region used to store global variables
 - ☐ some unknown value because it attempts to access the stack of a terminated thread
 - ☐ none of the above, explain: _____
- (d) (3 points) What is a possible final value of `x` (that might be printed out in `main`)?

Solution: 25